

Methodology: the logic behind every step

This document explains **what** each step of the pipeline does and **why** it was designed that way. Section numbers refer to the notebook.

Mapping to the project brief

Step in the brief	What the notebook does	Notebook section
1. Choose a relevant, web-scraped dataset	Scrape 3,000 newest Swiggy reviews + star ratings from Google Play	1
2. Clean and normalise	Remove duplicates, short and non-English reviews; slang, contractions, stopwords, lemmatisation	2
3. Feature extraction using TF-IDF	TF-IDF terms, distinctive terms, Logistic Regression, NMF topics	6
4. Tri-gram + sentiment analysis with sarcasm detection	VADER vs TextBlob vs RoBERTa; hybrid sarcasm detector; tri-grams	4, 5, 7
5. Unigram analysis + word cloud	Top unigrams; word clouds for all / negative / positive reviews	8
6. Interpret and suggest strategies	Aspect sentiment, pain-point ranking, insights, strategies with KPIs	9, 10, 11

Course outcome	Where it is met
CO1: approach tied to a marketing problem	Every step answers one of the four management questions in the README
CO2: right Python libraries for every step	See the tool choices below
CO3: recommendations for decision-making	Sections 10–11: ranked playbook with KPIs

Setup (configuration)

- **One configuration block.** Company, app ID, thresholds and model names live in one place, so the whole analysis can be repeated for a competitor by changing only `APP_ID`.
- **Self-installing.** Missing packages are installed automatically, so the notebook runs top to bottom on a fresh Colab.
- **Colour logic.** Sentiment is a polarity (bad ↔ good), so charts use a diverging pair: red = negative, grey = neutral, blue = positive. Blue/red stays readable for colour-blind readers, unlike red/green.

1. Data collection

What: the 3,000 newest English reviews from the Indian Play Store, via `google-play-scraper`.

Why this source and tool

- E-commerce sites (e.g. Amazon) block automated traffic (HTTP 503).
- Play Store reviews load through JavaScript, so `requests` + `BeautifulSoup` sees an empty page. `google-play-scraper` calls the same endpoint the browser uses and returns clean records.
- Every review comes with the customer's **own star rating**. This is a free, independent label used to (a) test which sentiment model is most accurate and (b) detect sarcasm (kind words + 1 star).

Why the newest reviews: they reflect the app as it is today; old reviews may describe problems already fixed.

Pagination: each call returns about 200 reviews plus a continuation token pointing to the next page. The loop pauses 1 second between requests to avoid overloading the server.

Reproducibility and privacy: raw data is saved to CSV and reused on re-runs. User names and profile pictures are dropped.

2. Cleaning and normalisation

Key design decision: two versions of every review

The two kinds of analysis need opposite preprocessing.

Version	Used by	Treatment	Why
<code>sentiment_text</code>	VADER, TextBlob, RoBERTa	Light: keeps capitals, "!!!", emojis, "not", "very"	These change the strength of a feeling. "NOT good!!!" is more negative than "not good".
<code>clean_text</code>	TF-IDF, n-grams, word clouds	Full: lowercase, contractions expanded, slang merged, no numbers/punctuation/stopwords, lemmatised	"delivered", "Delivery" and "deliver" should count as one idea.

Cleaning steps

Step	Rows after	Logic
Raw scrape	3,000	
Remove empty reviews / ratings	3,000	Nothing to analyse

Step	Rows after	Logic
Remove duplicates	1,762	A repeated review ID is scraping overlap; identical texts are copy-paste or one-word reviews ("good") that would inflate n-gram counts
Remove reviews under 3 words	1,264	"Good" has a rating but no <i>reason</i> ; this project is about <i>why</i> . Their average (4.16★) is reported so the removal is transparent
Remove Hindi / Hinglish	1,146	The Play Store language filter follows the phone's setting, not the review's language. English models score "bahut bekar app hai" as neutral. A review is kept only if $\geq 60\%$ of its words are recognised English
Final analysable reviews	1,145	

Normalisation choices

- **Slang dictionary:** "pls" → "please", "delivery boy / guy / executive" → "delivery partner". Without this, one idea is split across several n-grams and each looks smaller than it is.
- **Contractions expanded:** "don't" → "do not", so the negation survives as its own token.
- **Stopwords = standard list – negations + brand and filler words.** "not delivered" and "delivered" mean opposite things, so negations stay. "swiggy" and "app" appear everywhere and would dominate every word cloud without saying anything.
- **POS-aware lemmatisation of nouns and verbs only.** "delivered" → "deliver", "charges" → "charge". Adjectives are deliberately left alone, because WordNet maps "worst" → "bad", which would erase the customer's intensity.
- **Label from the stars:** 1-2★ = Negative, 3★ = Neutral, 4-5★ = Positive. This is the customer's own verdict and serves as ground truth.

3. Exploratory data analysis

- **Rating distribution:** shows how polarised customers are (58% gave 1★, 30% gave 5★).

- **Review length by rating:** median 21 words for 1★ vs 5 words for 5★. Long complaints are detailed diagnoses that the aspect analysis can use.
- **Company reply rate:** public replies are a cheap reputation lever, so the reply rate is a marketing metric in itself.

4. Sentiment analysis: three model families

Instead of trusting one tool, three approaches are compared against the star ratings.

Model	Type	Strength	Labelling rule
VADER	Rules + lexicon, built for social media	Understands emojis, CAPS, "!!!", negation, boosters	compound $\geq +0.05$ Positive, ≤ -0.05 Negative (thresholds from the VADER paper)
TextBlob	Pattern lexicon	Also measures subjectivity; simplest baseline	polarity $\geq +0.05$ / ≤ -0.05
RoBERTa	Transformer trained on ~124M tweets	Reads the whole sentence in context: handles "not bad at all" and long mixed reviews	highest of the three class probabilities

Why macro-F1 to pick the winner: it averages F1 over the three classes, so a model cannot score well by calling everything "Positive". Plain accuracy would reward that shortcut on an imbalanced dataset.

Result: RoBERTa won (macro-F1 0.67 vs 0.57 VADER vs 0.52 TextBlob). Lexicon models score each word on its own, so long reviews that mix complaints and praise confuse them. RoBERTa's output becomes the **literal sentiment**, which Section 5 then corrects.

Fallback: if the transformer cannot load (no internet or no PyTorch), the notebook continues with VADER and TextBlob.

5. Sarcasm detection

Sarcasm says the opposite of what is meant: "*Wow, amazing service, only 2 hours late!*" Every sentiment model sees "wow" and "amazing" and scores it positive, so real complaints are counted as praise.

Hybrid approach: a review is sarcastic if any one of three independent signals fires.

Signal	Rule	Logic	Fired
S1 Rating-text incongruity	Clearly positive text (score ≥ 0.30) + 1-2★ + mentions a real problem	Stars show true intent, text says the opposite. Requiring a real problem separates sarcasm from a mis-tap on the stars	8
S2 Cue phrases and emojis	"thanks for nothing", "great job", "wow", "so-called", 🤔 🤔, only in 1-2★ reviews	"Great job" in a 5★ review is sincere; in a 1★ review it is sarcasm	6
S3 Irony transformer	<u>cardiffnlp/twitter-roberta-base-irony</u> : probability ≥ 0.80 , rating ≤ 3 , text not already negative	Catches sarcasm without obvious cue words; trusted only when confident	11

Why combine three signals: each covers the others' blind spots. Rules are transparent but narrow; the model is broad but can be wrong; the mismatch check uses information (stars) that the text alone does not have.

Correction: a sarcastic review that the literal model called Positive or Neutral is re-labelled Negative (18 reviews). Six reviews with fully positive text, 1-2★ and no complaint were treated as star-rating mistakes, not sarcasm.

Caveat: S1 and S2 use the star rating, so part of the accuracy gain (86.2% → 87.4%) is by design. The real value is finding *which* complaints are voiced sarcastically.

6. Feature extraction with TF-IDF

TF-IDF = term frequency × inverse document frequency. A word scores high

when it is frequent in one review but rare across all reviews: "food" appears everywhere (low weight), "refund" marks a specific issue (high weight).

Setting	Value	Why
<code>ngram_range</code>	(1, 2)	Single words and two-word phrases such as "platform fee"
<code>min_df</code>	≥ 3 reviews	Drops typos and one-off noise
<code>max_df</code>	0.85	Drops terms in more than 85% of reviews; they carry no information
<code>sublinear_tf</code>	True	Uses $1 + \log(\text{tf})$, so one review repeating "worst" 10 times cannot dominate

TF-IDF is used four ways

Most important terms overall.

Distinctive terms: average TF-IDF in negative reviews minus positive ones.

Logistic Regression predicting happy (4–5★) vs unhappy (1–2★). Chosen because its coefficients are directly readable: they show which words drive bad ratings. A `Pipeline` re-learns the vocabulary inside each cross-validation fold to avoid data leakage. Result: 90.7% accuracy, macro-F1 0.90.

NMF topic modelling on negative reviews, to discover complaint themes without defining them in advance. NMF is used instead of LDA because it works directly on TF-IDF and gives cleaner topics on short texts. Its 6 themes cross-check the hand-built aspect dictionary in Section 9.

7. Tri-gram analysis

Single words lose context: "delivery" says nothing, while "delivery partner not" or "not deliver order" names the exact failure. Tri-grams (three-word sequences) surface these phrases.

Each phrase carries the **average star rating of the reviews containing it**, so it has a measured satisfaction cost (e.g. "delivery partner not" $\rightarrow$ 1.10★, "best food delivery" $\rightarrow$ 4.60★).

8. Unigrams and word clouds

Unigrams give the broad vocabulary. Three clouds (all, negative, positive) are drawn instead of one, because a single cloud mixes praise and complaints and hides both. Word clouds are mainly for communicating to non-technical stakeholders; the frequency tables carry the analysis.

9. Aspect-based sentiment

Overall sentiment says *how* customers feel; aspects say *why*. Each review is tagged with the business aspects it mentions, using a keyword dictionary built from Swiggy's value chain.

Metric	Meaning
Mention share	How many customers talk about the aspect (volume)
% negative	How badly it performs when they do (severity)
NSS	Net Sentiment Score = % positive – % negative
Pain score	Mention share × % negative. Ranks priorities by volume × severity
% sarcastic	Where customers are frustrated enough to turn sarcastic

Why volume × severity: an aspect that is 100% negative but mentioned by 0.7% of customers (Swiggy One) matters less than one that is 97% negative and mentioned by 22% (Refund & support). Ranking by either number alone would misdirect resources.

Also analysed: the most "helpful"-voted negative reviews (the Play Store shows these to every prospective user first), negative share over time, and sentiment by app version.

10–11. Interpretation and strategy

The findings text is **generated from the computed numbers**, not typed by

hand, so it stays correct whenever the data is refreshed.

Each top pain point is mapped to a concrete action and a KPI, so every recommendation is tied to a finding and to a way of measuring whether it worked.

Pain point	Action	KPI
Refund & customer support	Human agent within 2 chatbot turns; instant fraud-scored refunds for low-value missing items; refund-status timeline in the app	First-contact resolution %, refund turnaround time, support NSS
Delivery time	Realistic ETAs; proactive delay alerts; automatic coupon when the ETA is missed by > 10 min; more riders at peak hours in the worst pin codes	% orders within promised ETA, share of delay-related negative reviews
Pricing & fees	Show every fee before checkout; communicate their value; A/B-test fee bundling; cap fees for Swiggy One members	Checkout abandonment, fee-related complaint share

Cross-cutting strategies

- **Sarcasm-aware social listening:** combine a sentiment model with an irony model and a rating-text check in brand-monitoring dashboards.
- **Review response programme:** prioritise personalised replies to the most helpful-voted complaints.
- **Lead marketing with relative strengths** (offers and discounts) and avoid headlining the weakest aspect until operations back it up.
- **Monthly Voice-of-Customer KPI:** re-run the pipeline monthly and track NSS overall and by aspect (currently -31).